Heuristic Search (مقدمة)

📌 الجزء الأول: التعريف الأساسي

المفهوم الجوهري (Core Concept)

```text
Heuristic Search = استراتيجية بحث ذكية
تستخدم معرفة خاصة بالمجال (Domain Knowledge)
لتوجيه البحث نحو المسارات الواعدة
```

الفكرة من Newell & Simon:

```text
الذكاء = القدرة على اتخاذ "خيارات حكيمة"
في بيئة موارد محدودة
بحيث تظهر الحلول الصحيحة مبكراً
```

الأصل اللغوي + التعريفات

المصدر	التعريف	الملاحظة
الجذر اليوناني	eurisco = "أنا أكتشف"	نفس جذر "Eureka!" (أرخميدس)
George Polya	"دراسة طرق وقواعد الاكتشاف والاختراع"	التعريف الكلاسيكي
في AI (State Space)	**قواعد مصاغة لاختيار الفروع الواعدة في مساحة البحث**	التعريف التقني ✅

التعريف الامتحاني:

```text
Heuristic = قاعدة/دالة تقييم (Evaluation Function)
تُقدّر "الوعد" (Promise) لكل حالة/عقدة
تستخدم لترتيب أولويات البحث
```

📌 الجزء الثاني: متى نستخدم Heuristics؟

الحالتان الأساسيتان

الحالة	السبب	مثال تطبيقي	الخاصية
1. Inherent Ambiguities (غموض متأصل)	**لا يوجد حل دقيق وحيد** بسبب طبيعة البيانات	- التشخيص الطبي - Computer Vision - NLP	البيانات غامضة أو متعددة التفسير
2. Combinatorial Explosion (انفجار توافقي)	**الحل موجود لكن تكلفته حسابية ضخمة**	- الشطرنج - TSP - لعبة Go	فضاء البحث ينمو **أسياً** (Exponential)

التفصيل التقني

الحالة 1: Inherent Ambiguities

المبدأ:

```text
عندما تكون البيانات غامضة أو غير كاملة
→ لا يمكن الوصول لحل "مضمون"
→ نستخدم Heuristics لاختيار "الأكثر احتمالاً"
```

أمثلة:

المجال	الغموض	الـ Heuristic المستخدم
Medical Diagnosis	نفس الأعراض لعدة أمراض	قواعد احتمالية: "الأكثر شيوعاً أولاً"
Computer Vision	صورة قد تُفسّر بعدة طرق	قواعد السياق والتشابه
Speech Recognition	أصوات متشابهة	نماذج لغوية إحصائية

النتيجة:

- ✅ نصل لحل **معقول** (Reasonable) وليس **مثالي** (Optimal)
- ✅ الحل **مقبول عملياً** (Good Enough)

الحالة 2: Combinatorial Explosion

المبدأ:

```text
فضاء البحث ينمو بشكل أسي (b^d)
→ Exhaustive Search مستحيل عملياً
→ Heuristics تُقلّص الفضاء بـ "القص" (Pruning)
```

المعادلة الجوهرية:

```text
عدد الحالات = b^d
b = branching factor
d = depth

مثال: شطرنج
b ≈ 35, d ≈ 100
→ 100^35 حالة (أكبر من ذرات الكون!)
```

كيف يساعد Heuristic؟

```text
بدلاً من فحص كل الفروع:
1. قيّم كل فرع (Evaluate)
2. اختر الأفضل/الواعد (Select Best)
3. اقطع الباقي (Prune Bad Branches)

النتيجة: تقليص مساحة البحث بمقادير ضخمة
```

📌 الجزء الثالث: خصائص Heuristics (الحقائق الامتحانية)

✅ 1. Fallibility (قابلة للخطأ) 🔥

```text
المبدأ: Heuristic = تخمين مدروس (Informed Guess)
النتيجة: قد تخطئ وتقود لحل غير مثالي
```

الحقيقة الامتحانية:

```text
Heuristics لا تضمن:
❌ إيجاد الحل الأمثل (Optimal Solution)
❌ إيجاد حل أصلاً (قد تفشل)
✅ لكن تُحسّن الأداء العملي بشكل كبير
```

✅ 2. معلومات محدودة (Limited Information)

```text
المبدأ: الـ Heuristic تعتمد على الحالة الحالية فقط
النتيجة: لا تتنبأ بدقة بالمستقبل البعيد
```

مثال:

```text
في الشطرنج:
- Heuristic: "عدد القطع المتبقية"
- قد تُفضل حالة بقطع كثيرة
- لكن تؤدي لكش مات بعد 3 حركات!
```

✅ 3. محدودية متأصلة (Inherent Limitation)

```text
المبدأ: الـ Trade-off بين السرعة والدقة
النتيجة: نقبل "حل جيد" بدلاً من "حل مثالي"
```

الخاصية	Exhaustive Search	Heuristic Search
الدقة	✅ يضمن الحل الأمثل	🟡 قد لا يكون أمثل
السرعة	❌ بطيء جداً/مستحيل	✅ سريع عملياً
الذاكرة	❌ ضخمة	✅ أقل بكثير
الاكتمال	✅ يجد الحل إن وُجد	🟡 قد لا يجد

📌 الجزء الرابع: العلاقة بالنظم الخبيرة

Heuristics = Rules of Thumb

المبدأ:

```text
الخبراء البشريون يستخدمون "قواعد التجربة"
هذه القواعد تُستخلص وتُصاغ للنظام الخبير
```

العملية:

```text
1. Knowledge Acquisition (اكتساب المعرفة):
   مقابلة الخبراء البشريين

2. Knowledge Formalization (صياغة):
   تحويل "الحدس" → قواعد IF-THEN

3. Knowledge Representation (تمثيل):
   ترميز القواعد في النظام

4. Inference (استدلال):
   تطبيق القواعد للوصول للحل
```

أمثلة من Expert Systems:

النظام	المجال	الـ Heuristic المستخدم
MYCIN	تشخيص طبي	"إذا حمى + احمرار → احتمال عدوى بكتيرية"
DENDRAL	كيمياء	"ذرات متصلة بهذا الشكل → مركب من نوع X"
PROSPECTOR	جيولوجيا	"تكوينات صخرية معينة → احتمال وجود معادن"

الخاصية المشتركة:

```text
كل هذه قواعد استكشافية (Heuristic Rules)
- مبنية على خبرة
- قابلة للخطأ
- لكن فعّالة عملياً
```

📌 الجزء الخامس: المقارنة الوظيفية

Exhaustive vs Heuristic Search

الخاصية	Exhaustive (Blind)	Heuristic (Informed)
المعرفة المستخدمة	لا شيء (عمياء)	**معرفة خاصة بالمجال** ✅
استهلاك الوقت	❌ أسّي (Exponential)	✅ أقل بمراتب عديدة
استهلاك الذاكرة	❌ ضخم	✅ معقول
ضمان الحل الأمثل	✅ نعم (إن أكمل البحث)	❌ لا
ضمان إيجاد حل	✅ نعم (Complete)	🟡 ليس دائماً
التطبيق العملي	مشاكل صغيرة فقط	**مشاكل واقعية معقدة** ✅
أمثلة	BFS, DFS العادية	A*, Best-First, Hill Climbing

التأثير الكمي (بدون أرقام كبيرة)

المبدأ:

```text
Heuristic تُقلّص فضاء البحث بـ "مراتب عشرية"
(Orders of Magnitude)
```

مثال: Tic-Tac-Toe

الطريقة	حجم البحث النسبي	التحسين
كل الاحتمالات	9! (مئات الآلاف)	-
مع Symmetry	أقل بمرتبة	✅
مع Heuristic	**أقل بـ 4 مراتب عشرية**	✅✅✅✅

📌 الجزء السادس: حقائق امتحانية مركزة

🔥 10 حقائق لا يخرج عنها سؤال

1. التعريف

```text
Heuristic = قاعدة/دالة تقييم
تختار الفروع الواعدة (Promising)
في مساحة البحث
```

2. الأصل

```text
من اليونانية: eurisco = "أنا أكتشف"
نفس جذر Eureka!
```

3. متى نستخدمها؟

```text
حالتان:
1. Inherent Ambiguities (غموض البيانات)
2. Combinatorial Explosion (انفجار توافقي)
```

4. Fallibility

```text
Heuristics قابلة للخطأ (Fallible)
لا تضمن الحل الأمثل
قد تفشل تماماً
```

5. Trade-off

```text
نضحي بـ: الضمان (Guarantee)
نكسب: السرعة (Speed) والعملية (Practicality)
```

6. Pruning

```text
الآلية الرئيسية: القص (Pruning)
نتجاهل الفروع غير الواعدة
نركز على الواعدة
```

7. Expert Systems

```text
Heuristics = Rules of Thumb
قواعد خبرة مستخلصة من البشر
مصاغة كـ IF-THEN rules
```

8. Limited Information

```text
تعتمد على الحالة الحالية
لا تتنبأ بالمستقبل البعيد بدقة
```

9. Orders of Magnitude

```text
تُقلّص فضاء البحث بمقادير ضخمة
من مئات الآلاف → عشرات
(+4 مراتب عشرية في بعض الحالات)
```

10. الضرورة العملية

```text
في المشاكل الواقعية المعقدة:
Heuristics ليست خياراً، بل ضرورة
Exhaustive Search مستحيل عملياً
```

🎯ملخص شامل

1. 🔷التعريف والأصل(Definition & Origin)

- **المفهوم :**هو قاعدة أو دالة تقييم (**Evaluation Function**) تُستخدم لاختيار الفروع "الواعدة" في شجرة البحث بناءً على معرفة خاصة بالمجال.(**Domain Knowledge**)
- **الأصل اللغوي :**مشتق من الكلمة اليونانية **"eurisco"**وتعني "أنا أكتشف(**I discover**) "، وهو نفس جذر كلمة **."Eureka"**

---

2. 🔷متى نستخدم الـHeuristics ؟

نلجأ إليها عندما يكون الحل التقليدي (Exhaustive) مستحيلاً بسبب:

1. **الغموض المتأصل :(Inherent Ambiguities)** غموض البيانات كما في تشخيص الأمراض أو الرؤية الحاسوبية.
2. **الانفجار التوافقي :(Combinatorial Explosion)** نمو فضاء البحث بشكل أسي (**Exponential**) كما في لعبة الشطرنج.

---

3. 🔷الخصائص والمقايضة(Characteristics & Trade-off)

- **المقايضة :(Trade-off)** نضحي بالضمان (**Guarantee**) مقابل الحصول على السرعة (**Speed**) والعملية.
- **المميزات:** ✅
  - سريعة جداً من الناحية العملية.
  - تقلل فضاء البحث بمقدار "مراتب عشرية (**Orders of Magnitude**) "مثل 10 أو 100 ضعف.
- **العيوب:** ❌
  - قابلة للخطأ.(**Fallible**)
  - لا تضمن دائماً الوصول للحل الأمثل.(**Suboptimal**)

---

4. 🔷العلاقة بالنظم الخبيرة(Expert Systems)

في النظم الخبيرة، تُصاغ الـ Heuristics على شكل:

- **قواعد الخبرة :(Rules of Thumb)** قواعد مستمدة من خبرة البشر.
- **تمثيل المعرفة :**تُمثل عادةً بصيغة قواعد.(**IF-THEN rules**)

---

5. 🔷مقارنة سريعة(Comparison)

وجه المقارنة	البحث الكشفي(Heuristic/Informed)	البحث الشامل(Exhaustive/Blind)
الضمان	لا يضمن الحل الأمثل دائماً	يضمن الحل الأمثل (إذا اكتمل)
المعرفة	يعتمد على معرفة مسبقة بالمجال	يعتمد على البحث الأعمى
التكلفة	منخفضة وعملية	عالية جداً (وقت وذاكرة)
مثال	A* Search / Greedy Search	BFS / DFS

Heuristic Search Algorithms

📌 الجزء الأول: ربط الخوارزميات بهياكل البيانات

الجدول الأساسي (يخرج منه 40% من الأسئلة) 🔥

الخوارزمية	هيكل البيانات	الذاكرة	التراجع؟	الملاحظة
Hill-Climbing	**لا شيء** (Current State only)	✅ O(b) قليلة جداً	❌ **لا يوجد**	لا تحتفظ بتاريخ البحث
Best-First	**Priority Queue** (Open + Closed)	❌ O(b^d) كبيرة	✅ **موجود**	تحتفظ بكل المسارات البديلة
Dynamic Programming	**Matrix/Table** (للحلول الفرعية)	🟡 O(n×m) معتدلة	✅ نعم (عبر الجدول)	تخزين الحلول (Memoization)

التفصيل التقني

1. Hill-Climbing:

```text
Data Structure: لا شيء!
- تحتفظ فقط بـ Current State
- لا Open List
- لا Closed List
- لا تاريخ

النتيجة:
✅ ذاكرة قليلة جداً (Memory Efficient)
❌ لا يمكن التراجع (No Backtracking)
```

2. Best-First:

```text
Data Structures:
1. Open List = Priority Queue (sorted by f(n))
2. Closed List = Set/Hash Table (visited states)

النتيجة:
✅ يمكن التراجع (Backtracking Possible)
❌ ذاكرة كبيرة (Memory Intensive)
```

3. Dynamic Programming:

```text
Data Structure: 2D Matrix/Table
- كل خلية = حل لمشكلة فرعية
- تُملأ بالتدريج (Bottom-up أو Top-down)

المبدأ الوحيد المهم:
"تخزين الحلول الفرعية لإعادة الاستخدام"
(Avoid redundant computation)
```

📌 الجزء الثاني: تحليل فشل Hill-Climbing (جوهري!) 🔥🔥🔥

لماذا تفشل Hill-Climbing؟

السبب الأساسي:

```text
❌ لا يوجد Backtracking (لا تحتفظ بالبدائل)
❌ لا يوجد Memory (تنسى المسارات السابقة)
→ عمياء محلياً (Locally Blind)
```

المشاكل الثلاث الكلاسيكية

1. Local Maximum (القمة المحلية) 🔴

التعريف:

```text
حالة تكون أفضل من كل جيرانها المباشرين
لكنها ليست الحل الأمثل العام (Global Maximum)
```

لماذا تفشل الخوارزمية؟

```text
1. تصل Hill-Climbing للقمة المحلية
2. كل الجيران أسوأ منها
3. الخوارزمية تتوقف (ظناً أنها وصلت للحل)
4. لكن القمة العالمية (Global Max) في مكان آخر

المشكلة: لا يوجد Backtracking للعودة وتجربة مسار آخر!
```

رسم توضيحي (مفاهيمي):

```text
    Global Max
       /\
      /  \
     /    \_____ Local Max (هنا تتوقف!)
    /
___/
```

---

2. Plateau (الهضبة) 🔴

التعريف:

```text
منطقة مسطحة حيث كل الحالات المجاورة
لها نفس قيمة التقييم (Evaluation Value)
```

لماذا تفشل؟

```text
1. تصل الخوارزمية للهضبة
2. كل الاتجاهات متساوية (لا تحسن)
3. الخوارزمية تتوقف أو تتخبط عشوائياً
4. لا يوجد "اتجاه صعود" واضح

النتيجة: Stagnation (ركود/جمود)
```

3. Ridge (السلسلة الجبلية) 🔴

التعريف:

```text
سلسلة من القمم المحلية المتتالية
تحتاج لخطوات "نزول مؤقت" للوصول للقمة العالمية
```

لماذا تفشل؟

```text
1. للوصول للقمة العالية، يجب النزول أولاً
2. Hill-Climbing ترفض "النزول" (Downward moves)
3. تظل عالقة على قمة محلية في السلسلة

المشكلة: الخوارزمية "جشعة محلياً" (Locally Greedy)
```

رسم مفاهيمي:

```text
      Global Max
         /\
        /  \
       /    \
  /\  /
 /  \/  ← Ridge (سلسلة قمم)
```

جدول الفشل الشامل

المشكلة	الوصف	السبب	الحل البديل
Local Maximum	قمة محلية ≠ قمة عالمية	لا Backtracking	✅ Best-First
Plateau	منطقة مسطحة (لا تحسن)	لا اتجاه واضح	Random restart / Best-First
Ridge	يحتاج نزول مؤقت	ترفض النزول	Simulated Annealing / Best-First

📌 الجزء الثالث: تفكيك المعادلة f(n) = g(n) + h(n)

المعادلة الذهبية

```text
f(n) = g(n) + h(n)
```

المكون	الاسم	المعنى	الدور
g(n)	Actual Cost	**التكلفة الفعلية** من البداية إلى n	يمنع المسارات الطويلة
h(n)	Heuristic	**التقدير** من n إلى الهدف	يوجه نحو الهدف
f(n)	Total Estimate	**التقدير الكلي** للمسار	معيار الترتيب

ماذا يضيف g(n)؟ (سؤال امتحاني مهم!) 🔥

بدون g(n) (فقط h(n)):

```text
Problem: Greedy Search
- يتبع الاتجاه "الأقرب ظاهرياً" للهدف
- قد يغرق في مسار طويل جداً
- قد يدور في حلقة

مثال:
المسار A: طول 100، h=5
المسار B: طول 10، h=8
Greedy يختار A (لأن h أصغر) ← خطأ!
```

مع g(n):

```text
Solution: Balanced Search
- يوازن بين "طول المسار" و "القرب من الهدف"
- يعطي "طابع بحث بالعرض" (Breadth-first flavor)
- يمنع الغرق في مسارات طويلة

نفس المثال:
f(A) = 100 + 5 = 105
f(B) = 10 + 8 = 18
Best-First يختار B ← صحيح! ✅
```

الخصائص الناتجة

```text
g(n) > 0:
✅ يمنع الدوران في حلقات (Loops)
✅ يفضل المسارات القصيرة
✅ يضيف "Breadth-first character"
✅ ضروري لضمان الأمثلية في A*
```

📌 الجزء الرابع: المقارنة الوظيفية الشاملة

Hill-Climbing vs Best-First

الخاصية	Hill-Climbing	Best-First
هيكل البيانات	لا شيء (Current State)	Priority Queue (Open + Closed)
الذاكرة	✅ **قليلة جداً** O(b)	❌ **كبيرة** O(b^d)
التراجع	❌ **مستحيل**	✅ **ممكن**
ضمان الحل	❌ لا (يفشل عند Local Max)	✅ نعم (إذا موجود)
ضمان الأمثلية	❌ أبداً	🟡 فقط في A* (إذا h admissible)
السرعة	✅ سريعة	🟡 أبطأ
المشاكل	Local Max, Plateau, Ridge	Memory Explosion
الاستخدام	مشاكل بسيطة/محدودة	**مشاكل واقعية معقدة**
التشبيه	متسلق معصوب العينين	متسلق يحمل خريطة

متى نستخدم كلاً منهما؟

استخدم Hill-Climbing عندما:

```text
✅ الذاكرة محدودة جداً
✅ المشكلة بسيطة (لا local maxima كثيرة)
✅ حل "جيد" يكفي (ليس الأمثل)
✅ السرعة أهم من الدقة
```

استخدم Best-First عندما:

```text
✅ تحتاج ضمان الحل
✅ المشكلة معقدة (local maxima متعددة)
✅ الذاكرة متوفرة
✅ الأمثلية مهمة (استخدم A*)
```

📌 الجزء الخامس: Best-First كخوارزمية عامة

التنبيه الذكي 🔥

```text
Best-First Search = إطار عام (General Framework)
الخوارزميات المشهورة = حالات خاصة بناءً على f(n)
```

الحالات الخاصة

الخوارزمية	المعادلة	الخصائص	الاستخدام
Greedy Search	f(n) = **h(n)** فقط (g=0)	- سريعة - ❌ غير مثالية - قد تضيع	تطبيقات بسيطة
***A**	f(n) = **g(n) + h(n)** (h admissible)	- ✅ **مثالية** - ✅ كاملة - 🟡 ذاكرة كبيرة	**الأفضل عملياً**
Uniform Cost	f(n) = **g(n)** فقط (h=0)	- مثالية - بطيئة (مثل Dijkstra)	عندما لا heuristic
***Weighted A**	f(n) = g(n) + **w·h(n)** (w>1)	- أسرع من A* - قد تضحي بالأمثلية	توازن سرعة/جودة

الشرح التفصيلي

1. Greedy Search (الجشع):

```text
f(n) = h(n)   (تجاهل g تماماً)

المعنى:
- اختر الحالة "الأقرب ظاهرياً" للهدف
- تجاهل طول المسار

المشكلة:
❌ قد يختار مساراً طويلاً جداً (لأن h صغير)
❌ غير مثالي
```

2. A* (النجمة):

```text
f(n) = g(n) + h(n)
مع شرط: h(n) ≤ التكلفة الحقيقية (Admissible)

المعنى:
- توازن بين الطول والقرب
- h متفائلة (لا تبالغ في التقدير)

النتيجة:
✅ يضمن الحل الأمثل
✅ كامل
← الأفضل عملياً!
```

3. Uniform Cost:

```text
f(n) = g(n)  (تجاهل h تماماً)

المعنى:
- مثل Dijkstra
- لا معلومات استكشافية

الاستخدام:
عندما لا نملك heuristic جيد
```

العلاقة البيانية

```text
        Best-First Search
        (General Framework)
                |
      +-------+-------+
      |       |       |
   Greedy    A*    Uniform Cost
  (h only) (g+h)   (g only)
```

📌 الجزء السادس: Dynamic Programming (مختصر)

المبدأ الوحيد المهم

```text
Dynamic Programming:
"تخزين الحلول الفرعية (Subproblems) لإعادة الاستخدام"
← تجنب الحسابات المتكررة
```

الخصائص الأساسية:

- ✅ Data Structure: **Matrix/Table**
- ✅ المبدأ: **Memoization** (تخزين النتائج)
- ✅ التطبيق: مشاكل ذات تداخل في الحلول الفرعية
- ✅ مثال: Minimum Edit Distance، Shortest Path

ملاحظة امتحانية:

```text
DP ليست heuristic search بالمعنى الدقيق
لكنها خوارزمية تحسين (Optimization)
```

🎯 ملخص نهائي (احفظه!)

ربط الخوارزميات بالهياكل:

```text
Hill-Climbing: لا شيء (Current State)
Best-First:    Priority Queue + Closed Set
DP:            Matrix/Table
```

فشل **Hill-Climbing**:

```text
1. Local Maximum: أفضل محلياً ≠ أمثل عالمياً
2. Plateau: منطقة مسطحة (لا تحسن)
3. Ridge: يحتاج نزول مؤقت

السبب: ❌ لا Backtracking
```

المعادلة:

```text
f(n) = g(n) + h(n)

g(n): تكلفة فعلية (يمنع المسارات الطويلة)
h(n): تقدير (يوجه نحو الهدف)
```

المقارنة:

```text
Hill-Climbing:
✅ ذاكرة قليلة
❌ لا تراجع
❌ يفشل عند Local Max

Best-First:
✅ تراجع ممكن
✅ ضمان الحل
❌ ذاكرة كبيرة
```

Best-First كإطار عام:

```text
Greedy:       f = h
A*:           f = g + h  ← الأمثل
Uniform Cost: f = g
```

Heuristic Evaluation

📌 الجزء الأول: المعادلات الحاكمة (حفظ إجباري!) 🔥

المعادلة الأولى: المقبولية (Admissibility)

```text
h(n) ≤ h*(n)  لكل n
```

الرمز	المعنى	الملاحظة
h(n)	التقدير الاستكشافي (Heuristic Estimate)	ما تحسبه الدالة
h*(n)	التكلفة الحقيقية الفعلية (Actual Cost)	الواقع الحقيقي
≤	أقل أو يساوي (Never Overestimate)	**التقدير متفائل دائماً**

القاعدة الذهبية:

```text
إذا كانت h(n) ≤ h*(n) دائماً
→ الدالة Admissible
→ الخوارزمية A*
→ يُضمن الحل الأمثل ✅
```

📌 الجزء الثاني: قاعدة التضمين (القاعدة الماسية!) 💎

العلاقة الحاكمة

```text
Monotonic ⊂ Admissible
```

بالعربي:

```text
كل دالة رتيبة (Monotonic) هي مقبولة (Admissible)
لكن ليس كل دالة مقبولة رتيبة
```

الجدول المقارن

الخاصية	Admissible	Monotonic
الشرط الرياضي	$h(n) \le h^*(n)$	$h(n_i) - h(n_j) \le cost(n_i, n_j)$
ضمان الأمثلية	✅ نعم	✅ نعم (لأنها Admissible)
الاتساق المحلي	🟡 غير مضمون	✅ **مضمون**
إعادة فتح العقد	🟡 قد تحتاج	❌ **لا حاجة**
الكفاءة	🟡 جيدة	✅ **أفضل**
العلاقة	**الأعم** (أوسع)	**الأخص** (أضيق)

البرهان المنطقي

لماذا كل Monotonic هي Admissible؟

```text
إثبات:
1. افترض h رتيبة (Monotonic)
2. لأي مسار من n إلى الهدف:
   h(n) ≤ cost(n, n₁) + h(n₁)
   h(n₁) ≤ cost(n₁, n₂) + h(n₂)
   ...
   h(nₖ) ≤ cost(nₖ, goal) + h(goal)

3. بالجمع:
   h(goal) + [مجموع التكاليف] ≥ h(n)

4. وحيث أن h(goal) = 0 (عند الهدف)
   → h(n) ≤ h*(n)

5. ∴ h مقبولة (Admissible) ✅
```

لماذا العكس غير صحيح؟

```text
مثال مضاد:
يمكن أن تكون h(n) ≤ h*(n) عموماً
لكن في خطوة معينة:
h(nᵢ) - h(nⱼ) > cost(nᵢ, nⱼ)

مثال رقمي:
h(A) = 10, h(B) = 2, cost(A,B) = 3
الفرق: 10 - 2 = 8 > 3 ❌
→ ليست Monotonic
لكن قد تكون Admissible (إذا h*(A) ≥ 10)
```

سؤال MCQ متكرر 🔥

Which statement is TRUE?

- A) All admissible heuristics are monotonic
- ✅ B) **All monotonic heuristics are admissible**
- C) Admissible and monotonic are equivalent
- D) No relation between them

📌 الجزء الثالث: تحليل القوائم والرتابة

لماذا **Monotonicity** توفر الوقت؟

المبدأ:

```text
Monotonic Heuristic → No need to reopen Closed nodes
```

التحليل التفصيلي

مع دالة غير رتيبة (Non-Monotonic):

```text
المشكلة:
1. تكتشف العقدة n عبر مسار طويل
2. تضعها في Closed (ظناً أنك وجدت أقصر طريق)
3. لاحقاً، تجد مساراً أقصر لـ n
4. يجب إعادة فتحها (Reopen) وتوسيعها مجدداً

النتيجة:
❌ هدر وقت (processed twice or more)
❌ تعقيد إضافي في التنفيذ
```

مع دالة رتيبة (Monotonic):

```text
الميزة:
1. أول اكتشاف لـ n يكون عبر أقصر مسار مضمون
2. لا داعي للقلق من مسارات أقصر لاحقاً
3. بمجرد وضعها في Closed → انتهى الأمر

النتيجة:
✅ كل عقدة تُعالج مرة واحدة فقط
✅ كفاءة أعلى
✅ برمجة أبسط
```

المعادلة التفسيرية

```text
Monotonic → f(n) لا تنقص على طول المسار

التفصيل:
                    f(nⱼ) = g(nⱼ) + h(nⱼ)
g(nᵢ) + cost(nᵢ, nⱼ) + h(nⱼ) =
(بسبب الرتابة)  g(nᵢ) + h(nᵢ) ≤
                    f(nᵢ) =

∴ f قيمتها تزداد أو تبقى ثابتة (لا تنقص)
```

النتيجة:

```text
بما أن f تزداد:
→ أول عقدة نختارها من Open هي بالفعل الأقصر
→ لا حاجة لإعادة النظر فيها لاحقاً
```

الجدول المقارن (الوقت)

الخاصية	Non-Monotonic	Monotonic
معالجة العقدة	قد تُعالج **عدة مرات**	**مرة واحدة فقط** ✅
إعادة فتح Closed	✅ قد تحتاج	❌ لا حاجة
التعقيد الزمني	أسوأ	**أفضل** ✅
التعقيد البرمجي	أعقد	**أبسط** ✅

📌 الجزء الرابع: المقارنة الذكية (Informedness)

الشرط الرياضي

```text
h₂ أكثر معلوماتية من h₁ إذا:

∀n: h₂(n) ≥ h₁(n)
و
h₂(n) ≤ h*(n)  (كلاهما Admissible)
```

النتيجة المباشرة

```text
h₂ > h₁
→ h₂ أقرب للواقع (h*)
→ h₂ تقدير أدق
→ h₂ توسع عقد أقل
→ h₂ مساحة بحث أصغر ✅
```

المثال الكلاسيكي: **Puzzle-8**

الدالتان:

الدالة	التعريف	المثال	الخاصية
h_1: Misplaced Tiles	عدد القطع في غير مكانها	5 قطع خاطئة → h_1=5	✅ Admissible 🟡 معلوماتية أقل
h_2: Manhattan Distance	مجموع المسافات المانهاتنية	القطعة تحتاج 3 خطوات → تُضاف 3	✅ Admissible ✅ **أكثر معلوماتية**

المقارنة العملية:

```text
مثال: حالة معينة في puzzle-8

h₁ (Misplaced) = 4
h₂ (Manhattan) = 7
h* (الواقع) = 10

التحليل:
✅ h₁ ≤ h* (admissible)
✅ h₂ ≤ h* (admissible)
✅ h₂ > h₁ → h₂ أكثر معلوماتية

النتيجة:
- h₂ تعطي تقديراً أقرب (vs 10 7)
- h₁ تقدير أبعد (vs 10 4)
- استخدام h₂ يوسع عقد أقل
```

الدليل العملي:

المقياس	h_1 (Misplaced)	h_2 (Manhattan)
متوسط العقد الموسعة	~5000 عقدة	~300 عقدة ✅
الوقت	أطول	أسرع ✅
الدقة	تقدير بعيد	تقدير قريب ✅
التعقيد الحسابي	O(1) بسيط	O(n) أعقد قليلاً

القاعدة العامة

```text
كلما كانت h أكبر (وما زالت ≤ h*)
→ كانت أكثر معلوماتية (More Informed)
→ كان البحث أكفأ (Fewer nodes expanded)
```

الحد الأمثل:

```text
h*(n) هي الدالة المثالية (Perfect Heuristic)
لكنها غير عملية (لو عرفناها، لما احتجنا البحث!)
```

📌 الجزء الخامس: المفاضلة (Trade-off)

القاعدة الذهبية الوحيدة

```text
More Informed Heuristic ⇄ Higher Computation Cost
```

بالتفصيل:

```text
h أكثر معلوماتية:
  ✅ توسع عقد أقل → توفير وقت البحث
  ❌ حسابها أعقد → استهلاك وقت CPU

h أقل معلوماتية:
  ✅ حسابها سريع
  ❌ توسع عقد أكثر → هدر وقت البحث
```

نقطة التوازن

العامل	التأثير
وقت حساب h(n)	يُضرب في **كل** عقدة
عدد العقد الموسعة	ينقص مع h أكثر معلوماتية
الوقت الكلي	= (وقت h × عدد العقد) + overhead

القرار:

```text
إذا: (الوقت الموفر في البحث) > (الوقت الزائد في الحساب)
→ استخدم h الأكثر معلوماتية ✅

إذا: h معقدة جداً (تستهلك وقت كبير)
→ قد تكون h الأبسط أفضل عملياً
```

مثال عملي: Puzzle-8

```text
:h₁ (Misplaced)
  - وقت الحساب: 0.001 ms
  - عدد العقد: 5000
  - الوقت الكلي: 5000 × 0.001 = 5 ms

:h₂ (Manhattan)
  - وقت الحساب: 0.003 ms (أعقد 3 مرات)
  - عدد العقد: 300 (أقل بـ 16 مرة)
  - الوقت الكلي: 300 × 0.003 = 0.9 ms

النتيجة: h₂ أسرع بـ 5× رغم تعقيدها! ✅
```

الخلاصة الذكية

```text
الهدف: تقليل الوقت الكلي، وليس تقليل وقت h فقط

المعادلة التقريبية:
Total Time ≈ (Nodes Expanded) × (Time per h)

القاعدة:
اختر h التي تُقلل الحاصل الكلي
```

📌 الجزء السادس: حقائق A* الذهبية (5 حقائق لا يخرج عنها سؤال)

✅ 1. التعريف المزدوج

```text
A* = Algorithm A + Admissible Heuristic

بالتفصيل:
Algorithm A: أي Best-First يستخدم f=g+h
Algorithm A: A* حيث h(n) ≤ h*(n)
```

الحقيقة الامتحانية:

```text
كل A* هي Algorithm A
ليس كل Algorithm A هي A* (إلا إذا h admissible)
```

✅ 2. ضمان الأمثلية (Optimality Guarantee)

```text
إذا كانت h admissible:
→ A* تضمن إيجاد الحل الأمثل (Optimal Solution)
```

البرهان المختصر:

```text
لأن h متفائلة (لا تبالغ):
→ A* لن تتجاهل مساراً قصيراً
→ لن تتوقف قبل فحص كل المسارات الأفضل
→ أول حل تجده هو الأقصر ✅
```

✅ 3. الكفاءة المثلى (Optimal Efficiency)

```text
A* توسع أقل عدد ممكن من العقد
(مقارنة بأي خوارزمية أخرى تضمن الأمثلية)
```

القاعدة:

```text
لا توجد خوارزمية أخرى (تستخدم نفس h):
- تضمن الحل الأمثل
- وتوسع عقد أقل من A*

A* = المعيار الذهبي (Gold Standard) ✅
```

✅ 4. الاكتمال (Completeness)

```text
A* كاملة (Complete):
إذا كان الحل موجوداً، ستجده حتماً
```

الشروط:

```text
✅ b (branching factor) محدود
✅ تكلفة الخطوات > 0 (لتجنب الحلقات)
✅ h admissible
```

✅ 5. الرتابة = كفاءة عليا

```text
إذا كانت h monotonic (إضافة لـ admissible):
→ A* لا تحتاج إعادة فتح Closed nodes
→ كفاءة أعلى
→ تنفيذ أبسط
```

الخلاصة:

```text
A* مع h monotonic = الأمثل عملياً
- ضمان الحل الأمثل ✅
- أقل عقد موسعة ✅
- لا إعادة معالجة ✅
- برمجة أبسط ✅
```

الجدول الذهبي لـ A*

الخاصية	A*	ملاحظة
Optimal	✅ نعم	إذا h admissible
Complete	✅ نعم	إذا b محدود
Optimally Efficient	✅ نعم	أقل عقد ممكنة
Memory	❌ O(b^d)	المشكلة الوحيدة
Time	🟡 يعتمد على h	كلما h أفضل، الوقت أقل

🎯 ملخص نهائي (احفظه!)

المعادلات الثلاث:

```text
1. Admissibility: h(n) ≤ h*(n)
2. Monotonicity:  h(nᵢ) - h(nⱼ) ≤ cost(nᵢ, nⱼ)
3. Informedness:  h₂ > h₁ (both ≤ h*)
```

قاعدة التضمين:

```text
Monotonic ⊂ Admissible
كل رتيبة مقبولة ✅
العكس غير صحيح ❌
```

الرتابة والقوائم:

```text
Monotonic → لا إعادة فتح Closed
→ كل عقدة تُعالج مرة واحدة
→ كفاءة أعلى
```

المقارنة:

```text
h₂ > h₁ → h₂ أكثر معلوماتية
→ عقد أقل
→ بحث أسرع
مثال: Manhattan > Misplaced
```

المفاضلة:

```text
h معقدة ⇄ بحث أسرع
الهدف: تقليل الوقت الكلي
```

حقائق A*:

```text
1. A* = Algorithm A + Admissible h
2. تضمن الأمثلية ✅
3. أقل عقد ممكنة (Optimally Efficient)
4. كاملة (Complete)
5. مع Monotonic → أكفأ
```

Expert Systems & AND/OR Graphs

📌 الجزء الأول: Rules as Heuristics (لماذا؟)

السؤال الجوهري

```text
لماذا قواعد النظم الخبيرة = Heuristics
وليست Logical Truths؟
```

الإجابة المركزة

الخاصية	Logical Truth	Heuristic Rule (Expert System)
اليقين	✅ صحيحة بنسبة 100%	🟡 **احتمالية/غير مؤكدة**
المصدر	رياضيات، منطق صوري	**خبرة بشرية** (Rules of Thumb)
المثال	2+2=4	"إذا دخل جيد + مدخرات → استثمر في أسهم"
الاستثناءات	❌ لا توجد	✅ **توجد** (قد يفضل شخص الأمان)
القابلية للخطأ	❌ لا تخطئ	✅ **قابلة للخطأ** (Fallible)
التمثيل	True/False فقط	**Certainty Factor** (درجة ثقة)

الحقيقة الامتحانية 🔥

```text
قواعد النظم الخبيرة = Heuristics لأنها:
1. مستخلصة من تجارب بشرية (ليست بديهيات)
2. تعمل "عادةً" وليس "دائماً"
3. تحتمل الاستثناءات
4. تتعامل مع عدم اليقين (Uncertainty)
5. قابلة للتحديث بناءً على نتائج جديدة

∴ Rules = Informed Guesses (تخمينات مدروسة)
```

📌 الجزء الثاني: AND/OR Graphs (التعريف التقني)

Hypergraph (الرسم الفائق)

```text
AND/OR Graph = نوع خاص من Hypergraph
```

المفهوم	التعريف
Hypergraph	رسم بياني حيث القوس (Hyperarc) يربط عقدة بـ **مجموعة** من العقد (وليس عقدة واحدة)
AND Relation	**يُمثل بـ Hyperarc** يربط Parent بمجموعة Children (كلهم ضروريون)
OR Relation	يُمثل بأقواس منفصلة (أحدهم يكفي)

التمثيل البصري

OR Node:

```text
      Parent
       /  \
    C₁    C₂

لإثبات Parent: أثبت C₁ OR C₂
(أحدهما يكفي)
```

AND Node:

```text
      Parent
       /⌒\  (قوس منحنى = Hyperarc)
    C₁    C₂

لإثبات Parent: أثبت C₁ AND C₂
(كلاهما ضروري)
```

العلاقة بالنظم الخبيرة

```text
Rule: IF (condition1 AND condition2) THEN conclusion

يُمثل كـ:
- conclusion = Parent node
- conditions = Children nodes
- الربط بينهم = Hyperarc (AND relation)
```

📌 الجزء الثالث: معايير النجاح (Success Criteria) 🔥

القاعدة الذهبية

نوع العقدة	شرط النجاح	التفسير
OR Node	نجاح **أحد** الأبناء على الأقل	أي مسار ناجح يكفي
AND Node	نجاح **جميع** الأبناء	كل المسارات يجب أن تنجح

التفصيل

OR Node Success:

```text
Parent OR-node succeeds IF:
∃ child: child succeeds

بالعربي:
يكفي نجاح ابن واحد فقط
الباقي لا يهم
```

مثال:

```text
Goal: إثبات "استثمار جيد"

OR branches:
- استثمار في أسهم ✅ (نجح)
- استثمار في عقارات ❌ (فشل)
- استثمار مختلط ❌ (فشل)

النتيجة: Goal ناجح ✅ (لأن فرع واحد نجح)
```

AND Node Success:

```text
Parent AND-node succeeds IF:
∀ children: child succeeds

بالعربي:
يجب نجاح كل الأبناء
فشل أي ابن = فشل كامل
```

مثال:

```text
Rule: IF (income_good AND savings_adequate) THEN invest_stocks

لإثبات invest_stocks:
- income_good ✅ (نجح)
- savings_adequate ❌ (فشل)

النتيجة: القاعدة فشلت ❌ (لأن شرط واحد فشل)
```

جدول المقارنة السريع

الخاصية	OR Node	AND Node
شرط النجاح	≥ 1 ابن ناجح	**جميع** الأبناء ناجحون
شرط الفشل	كل الأبناء فشلوا	**أي** ابن فشل
الاستراتيجية	جرّب حتى تجد ناجحاً	يجب فحص الكل
التوقف المبكر	✅ عند أول نجاح	✅ عند أول فشل

📌 الجزء الرابع: Certainty Factors (عوامل اليقين)

جدول القيم الأساسي

القيمة	المعنى	التفسير
1.0+	يقين تام (Certainly True)	الاستنتاج صحيح بنسبة 100%
0.8+	ثقة عالية	احتمال كبير جداً
0.5+	ثقة متوسطة	احتمال معقول
0.2+	ثقة ضعيفة	احتمال منخفض
0.0	غير معروف (Unknown)	لا دليل لصالح أو ضد
0.2-	شك ضعيف	احتمال ضعيف أن يكون خطأ
0.5-	شك متوسط	على الأرجح خطأ
0.8-	شك قوي	احتمال كبير أن يكون خطأ
1.0-	يقين بالخطأ (Certainly False)	خطأ بنسبة 100%

التطبيق في Expert Systems

مثال: النصيحة المالية

```text
:Input
- income = good (CF = 0.9)
- savings = adequate (CF = 0.7)

Rule: IF income_good AND savings_adequate THEN invest_stocks

:Output
invest_stocks with CF = min(0.9, 0.7) = 0.7
(نستخدم الأدنى في AND)
```

قواعد الدمج (Combination Rules)

العملية	القاعدة	مثال
AND	$CF = \min(CF_1, CF_2)$	min(0.9, 0.7) = 0.7
OR	$CF = \max(CF_1, CF_2)$	max(0.8, 0.5) = 0.8
NOT	CF = -CF	-(0.7) = -0.7

📌 الجزء الخامس: Pruning with Threshold (تقليم البحث)

المبدأ الأساسي

```text
استخدام Threshold (عتبة الثقة) لتقليل مساحة البحث
= تجاهل القواعد/النتائج ذات الثقة المنخفضة
```

الآلية

تحديد Threshold:

```text
Threshold = 0.3 (مثال)

إذا CF لقاعدة أو نتيجة < 0.3
→ تُهمل تماماً (Pruned)
→ لا تُفحص أبناؤها
```

المثال العملي

```text
Goal: نصيحة استثمارية

القواعد المتاحة:
1. invest_stocks    → CF = 0.8 ✅ (> threshold)
2. invest_bonds     → CF = 0.5 ✅ (> threshold)
3. invest_gold      → CF = 0.2 ❌ (< threshold) → Pruned
4. invest_crypto    → CF = 0.1 ❌ (< threshold) → Pruned

النتيجة:
- نفحص فقط الخيارين 1 و 2
- نوفر 50% من الوقت
```

استراتيجيات الـ Pruning

الاستراتيجية	الوصف	الاستخدام
Hard Pruning	حذف كل ما < Threshold	تطبيقات عامة
Soft Pruning	تأخير الفحص (أولوية منخفضة)	مجالات حساسة
Best-First	فحص الأعلى CF أولاً	تحسين السرعة
Beam Search	الاحتفاظ بأفضل N فقط	ذاكرة محدودة

الفوائد

```text
✅ تقليل مساحة البحث (Search Space Reduction)
✅ توفير الوقت (Faster Response)
✅ تقليل الأسئلة للمستخدم (في Interactive Systems)
✅ تركيز على الاحتمالات القوية فقط
```

📌 الجزء السادس: الميكانزم الكامل (الخوارزمية)

خطوات البحث في Expert System

```text
1. Start with Goal (backward) أو Facts (forward)

2. For each applicable rule:
   a. Calculate CF for premises (conditions)
   b. IF CF < Threshold → Prune (تجاهل)
   c. ELSE: Apply rule, calculate conclusion CF

3. For AND nodes:
   - Check ALL children
   - Success IF all succeed
   - CF = min(CF_children)
   - Early stop IF any < Threshold

4. For OR nodes:
   - Check children one by one
   - Success IF any succeeds
   - CF = max(CF_children)
   - Early stop IF one > high threshold

5. Return conclusion(s) with CF values
```

المقارنة مع Puzzle Search

الخاصية	Puzzle Search	Expert System
Heuristic	دالة واحدة (h)	**قواعد متعددة** مع CF
اليقين	حالات محددة	**احتمالية** (CF values)
الهدف	حالة نهائية واضحة	**استنتاج** مع درجة ثقة
البحث	في مساحة حالات	**في شجرة قواعد** (AND/OR)
Pruning	بناءً على f(n)	بناءً على **CF & Threshold**

🎯 ملخص نهائي (احفظه!)

Rules as Heuristics:

```text
- مستخلصة من خبرة بشرية
- قابلة للخطأ (Fallible)
- احتمالية (ليست يقينية)
- تتعامل مع عدم اليقين
```

AND/OR Graphs:

```text
Hypergraph: قوس يربط عقدة بمجموعة
AND = Hyperarc (كلهم ضروريون)
OR = أقواس منفصلة (أحدهم يكفي)
```

Success Criteria:

```text
OR: نجح ≥ 1 ابن
AND: نجح كل الأبناء
```

Certainty Factors:

```text
+1.0 = يقين تام
 0.0 = غير معروف
-1.0 = خطأ تام

AND → min(CF)
OR  → max(CF)
```

Pruning:

```text
IF CF < Threshold → Prune
توفير الوقت والموارد
```